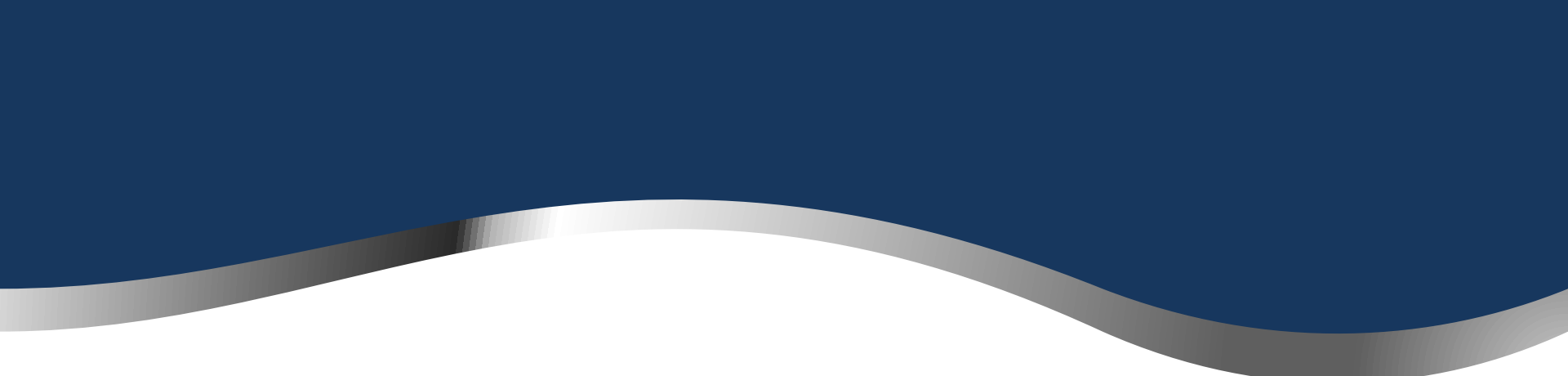


A decorative wavy line in dark blue and grey spans the top of the slide.

ABAP Objects

Course Outline

- Unit 1: Overview
- Unit 2: Concept
- Unit 3: Keywords and Syntax
- Unit 4: Exception Handling
- Unit 5: Use of ABAP Objects - Advanced Concept
- Unit 6: ABAP Workbench Tool- Global Class
- Unit 7: OO Transaction
- Unit 8: The Control Framework
- Unit 9: BAdIs

Unit 1: Overview

Overview

ABAP Objects

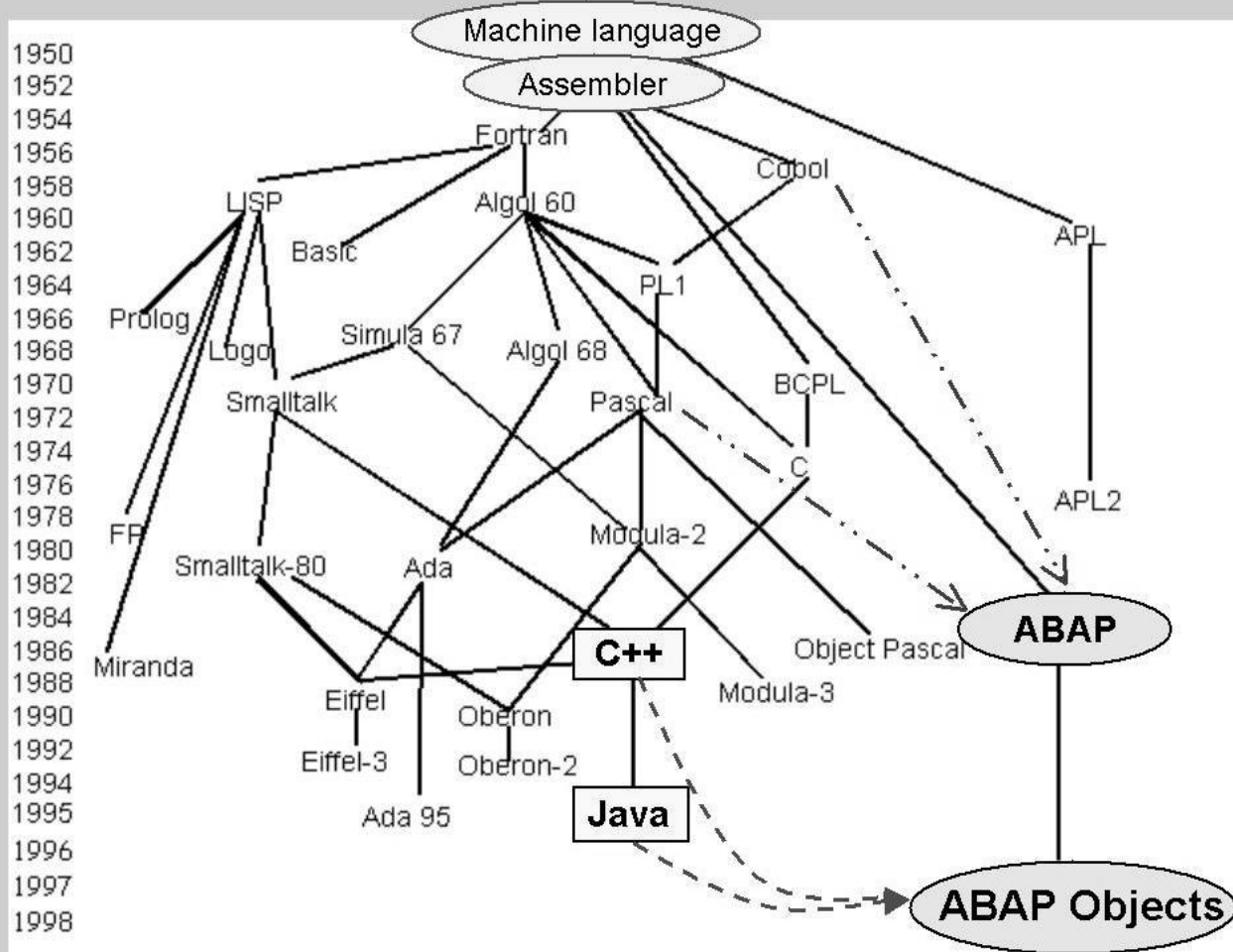
ABAP

- **A**dvanced **B**usiness **A**pplication **P**rogramming
- Is a historically grown 4GL language
- Is specialized for business applications

ABAP Objects

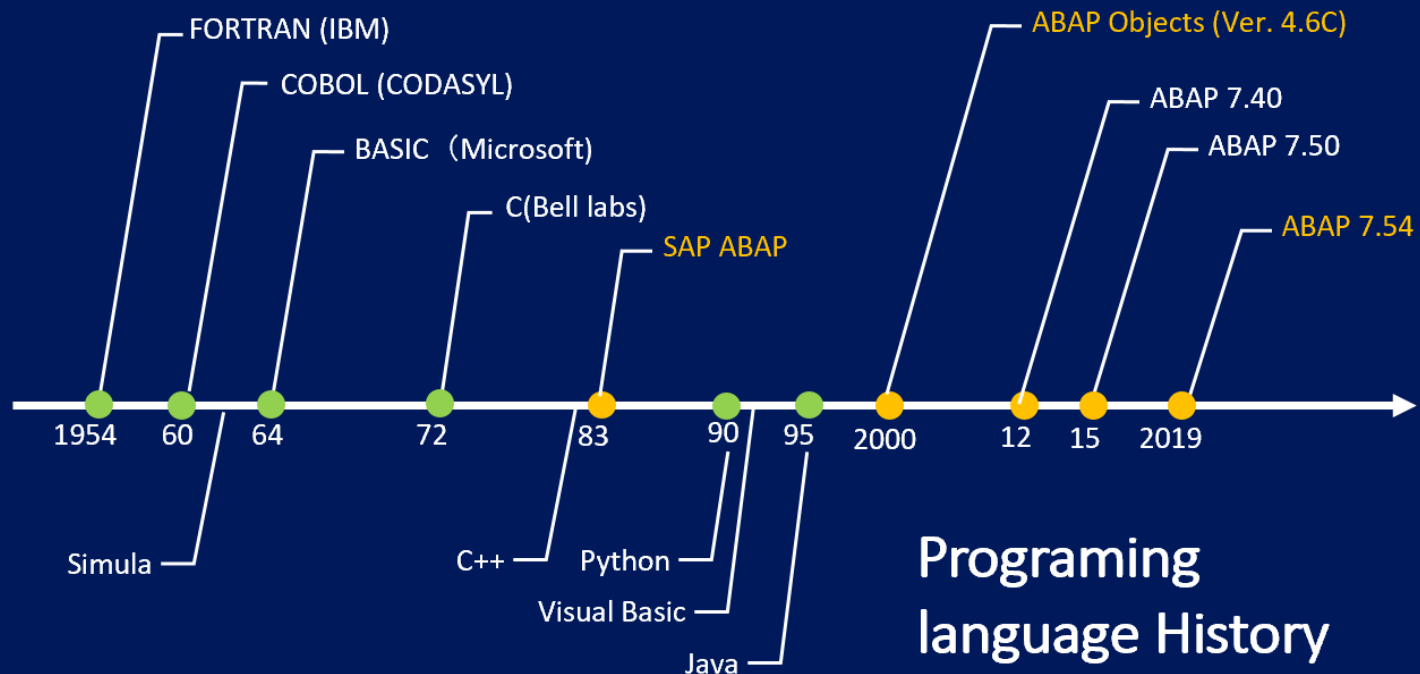
- Object-oriented extension of ABAP programming language

History



History

Time Line of Programming Language



Programing
language History

ABAP Programming Model

ABAP supports a hybrid programming model:

- A object-oriented programming model that is based on classes and interface of ABAP objects
- A procedural programming model that is based on procedural calls and system event handling
- Both models are interoperable – you can call classes from classic procedures, and you can call classic procedures from methods.

Unit 2: Concept

Concept

Object-oriented Programming

Abstraction

- Ability to reflect real-world processes
- real-world modeled in classes and mapped in objects

Encapsulation

- Implementation details are hidden behind interfaces.
- Ensure that the abstract representation of an object is used only in accordance with its specification.

Inheritance

- Abstractions derived from existing ones.

Instantiation

- Enables you to create multiple instances of a class.

Why use ABAP Objects?

- ABAP Objects results in a clean and secure programming style.
- SAP delivers essential APIs (ex. Control Framework) in object-oriented wrapped classes.
- Most SAP new products are written in ABAP Objects!

Object-oriented Programming Basics

- Class is the model or the template
- Object is an instance *of class*
- **Local class is created within any ABAP program and only visible there**
- **Global class is created with the Class Builder tool and visible in any ABAP program**

Unit 3: Keywords and Syntax

Keywords and Syntax

Define a CLASS

CLASS class_name DEFINITION.

....

ENDCLASS.

CLASS class_name IMPLEMENTATION.

....

ENDCLASS.

Visibility Section

Public

- All components are public and can be addressed by all subclasses, the class itself and other external classes.

Protected

- Components of this section are protected and can only be addressed by the class itself and the subclasses

Private

- Can only be used in the methods of the class itself

Define Visibility Section

CLASS class_name DEFINITION.

PUBLIC SECTION.

....

PROTECTED SECTION.

....

PRIVATE SECTION.

....

ENDCLASS.

Controlling the Instantiation

Public instantiation

- Allow to any user to create objects (default)

Protected instantiation

- Allow the creation of objects in methods of subclasses

Private instantiation

- Only the class itself can create objects

Define Instantiation

CLASS class_name

DEFINITION CREATE

PUBLIC | PROTECTED | PRIVATE.

ENDCLASS.

Components

Components of a class

1. Attributes
2. Methods
3. Events

Components – Attribute

- The data objects within a class
- All data types of the ABAP type hierarchy can be used
- 'DATA' statement is used for instance attributes
- 'CLASS-DATA' for static attribute

Define Attributes

CLASS attributes DEFINITION.

PRIVATE SECTION.

DATA objectValue TYPE i.

CLASS-DATA objectCount TYPE i.

ENDCLASS.

Components – Methods

- Processing block with a parameter interface
- Local data
- Instance method can access all attributes and events
- Static method can access only static attributes and events

Define Instance Methods

CLASS class_name DEFINITION.

PUBLIC SECTION.

METHODS meth

IMPORTING IM_ TYPE type ..

EXPORTING EX_ TYPE type ..

CHANGING CH_ TYPE type ..

RETURNING VALUE(RE_ TYPE type ..)

RAISING <Exception Class>.

ENDCLASS.

Define Static Methods

CLASS class_name DEFINITION.

PUBLIC SECTION.

CLASS-METHODS class_meth

IMPORTING IM_ TYPE type..

EXPORTING EX_ TYPE type ..

CHANGING CH_ TYPE type ..

RETURNING VALUE(RE_ TYPE type

RAISING <Exception Class>.

ENDCLASS.

Implementation of Methods

```
CLASS class_name IMPLEMENTATION.  
    METHOD meth.  
    ...  
    ENDMETHOD.  
ENDCLASS.
```


Components – Methods

Two special methods

constructor

- Implicitly called for each instantiation of a new object
- Execute by runtime environment.

class_constructor

- First time a class is accessed

Destructors

ABAP Objects *does not* provide any destructors for application development.

From ABAP 7.51 (Internal Use Only)

Define Object Reference Variables

To create and access an object, object references in reference variables are needed.

Syntax:

DATA ref TYPE REF TO class

Instantiating

Syntax:

CREATE OBJECT ref.

Object = NEW #() or Object = NEW class().

Different Selectors

Accessing fields of a structure

- Structure component selector “-”

Accessing object components

- Object component selector “->”
- Class component selector “=>”
- Interface component selector “~”

Use of Methods

Call instance method:

Syntax:

CALL METHOD oref->get_data. or oref->get_data().

Call class method:

Syntax:

CALL METHOD class=>get_data. or class=>get_data ().

Functional Method

Functional method

- Method with any number of input parameters but only one return value and exporting parameters.
- It can be used in operand positions for functions and expressions.

Use of Functional Method

Functional method used in expressions:

Syntax

```
IF obj->functional_method( ) = abap_true.  
    ..... do some thing  
ENDIF.
```

Method Call

CALL METHOD oref->get_data.

CALL METHOD class=>get_data.

CALL METHOD oref->get_data

IMPORTING P1 = P1

RECEIVING P2 = P2.

Method Call Short Form

- Functional Method Call Style

oref->get_data().

class=>get_data().

P2 = oref->get_data(P1).

Special Reference Variables

The self-reference:

- `me`

The superclass-reference(used in method call):

- `super`

CLASS DEFINITION DEFERRED

CLASS – DEFERRED.

Syntax

*CLASS class DEFINITION
DEFERRED.*

CLASS DEFINITION DEFERRED

CLASS LCL_ENGINE DEFINITION DEFERRED.

CLASS LCL_CAR DEFINITION.

PUBLIC SECTION.

DATA GO_ENGINE TYPE REF TO LCL_ENGINE.

ENDCLASS.

CLASS LCL_ENGINE DEFINITION.

PUBLIC SECTION.

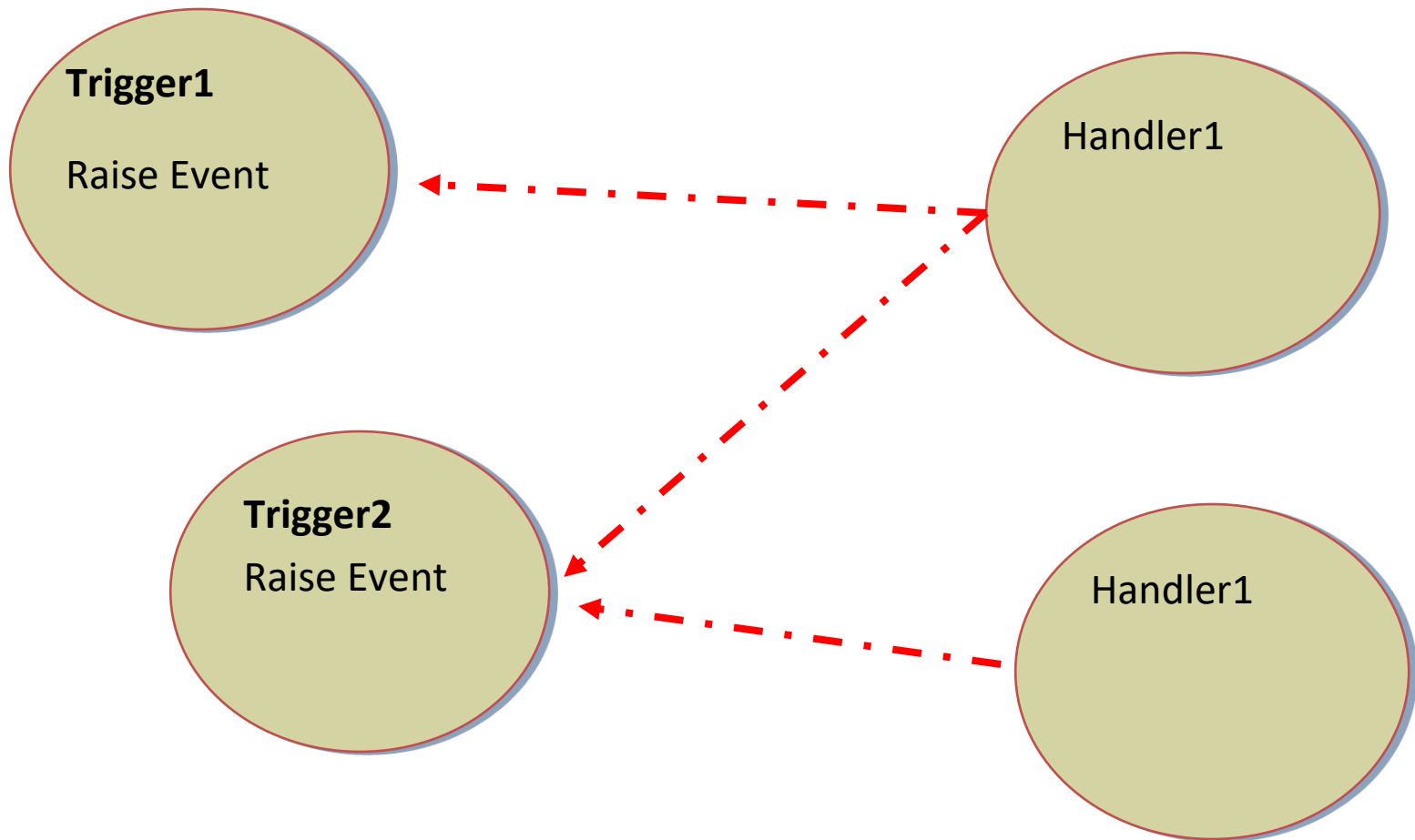
DATA GO_CAR TYPE REF TO LCL_CAR.

ENDCLASS.

Components – Events

- **Allow the objects of a class to publish its status. Other objects can then respond to the change in status.**
 1. Event trigger does not know the event handler.
 2. Event trigger therefore does not initially know if the event will even have an effect.
- **Publish and Subscribe mechanism, i.e. Broadcasting**

Event Triggers and Event Handlers



Terms

Events

Each method can trigger the events of its class

Handler methods

Methods can be declared as handler methods for events

Registering handlers

Objects for an event can be registered at runtime as handlers for the objects of the classes which can trigger the event

Define Events

CLASS raising_class DEFINITION.

PUBLIC SECTION.

EVENTS: raised_event.

ENDCLASS.

Raise Events

```
CLASS raising_class IMPLEMENTATION.  
    METHOD raising_method.  
        RAISE EVENT raised_event.  
    ENDMETHOD.  
ENDCLASS.
```

Define Handler Methods

CLASS handler_class DEFINITION.

PUBLIC SECTION.

*METHODS: handler_method FOR EVENT
raised_event OF raising_class.*

ENDCLASS.

Handler Methods

```
CLASS handler_class IMPLEMENTATION.  
    METHOD handler_method.  
        ... some logics here  
    ENDMETHOD.  
ENDCLASS.
```

Register Handlers

SET HANDLER

handler_instance->handler_method
FOR ALL INSTANCES.

SET HANDLER

handler_instance->handler_method
FOR handled_instance.

Unit 4: Exception Handling

Exception Handling

Exception Handling

Exceptions are events that arise during the execution of an ABAP program at which the program is interrupted, because it is not possible to continue processing the program in a meaningful way.

Exceptions are either treatable or untreatable.

Exception Handling

Runtime Errors

Exceptions Before Release 6.10

Class-Based Exceptions

Runtime Errors

Exception that was not handled

data RESULT type I.

RESULT = 1 / 0.

Runtime Errors

Shortdump screenshot

The screenshot shows the 'Runtime Error - Description of Exception' dialog box in SAP. The menu bar includes 'Runtime Errors', 'Edit', 'Goto', 'System', and 'Help'. The toolbar contains various icons for file operations and help. The dialog has two tabs: 'Long Text' and 'Debugger'. The 'Long Text' tab is active, showing a table with error details.

Runtime Errors	COMPUTE_INT_ZERODIVIDE
Except.	CX_SY_ZERODIVIDE
Date and Time	2008/11/04 16:55:04

Below the table, there are two sections with blue headers:

Short text
Division by 0 (type I)

What happened?
Error in the ABAP Application Program

Exceptions Before Release 6.10

Handling exceptions as catchable runtime errors

```
data RESULT type I.  
catch system-exceptions ARITHMETIC_ERRORS = 4.  
  RESULT = 1 / 0.  
endcatch.  
if SY-SUBRC = 4.  
  ...  
endif.
```

Class-Based Exceptions

Handling exceptions using/with exception classes

data MYREF type ref to CX_SY_ARITHMETIC_ERROR.

data ERR_TEXT type STRING.

data RESULT type I.

try.

RESULT = 1 / 0.

catch cx_sy_arithmetic_error into MYREF.

ERR_TEXT = MYREF->GET_TEXT().

endtry.

Defining Exceptions

All exception classes must inherit from the common superclass **CX_ROOT** and one of its subordinate classes:

- **CX_STATIC_CHECK**
- **CX_DYNAMIC_CHECK**
- **CX_NO_CHECK**

Propagating Exceptions

The **RAISING** addition is specified in the declaration of a method / subroutine:

METHODS meth ... RAISING cx_... cx_...

FORM form ... RAISING cx_... cx_...

Unit 5: Use of ABAP Objects - Advanced Concept

Use of ABAP Objects - Advanced Concept

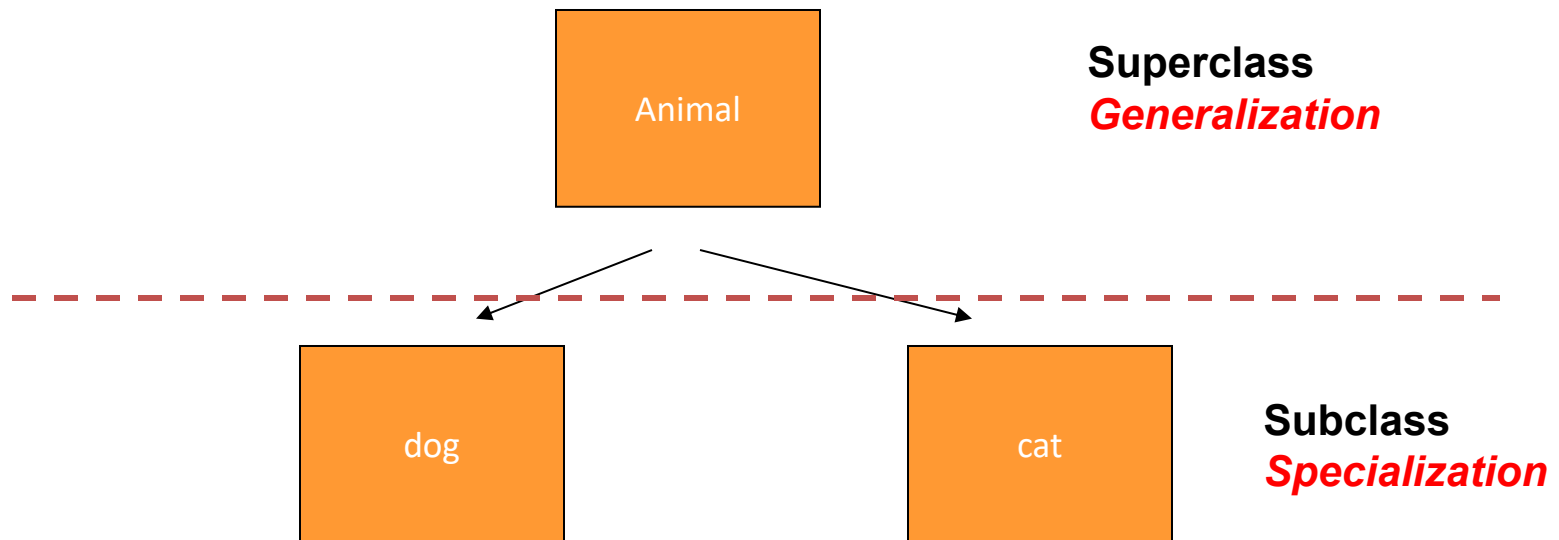
Advanced Concept of Object-oriented Programming

Inheritance

Interface

Friend

Inheritance



Visibility in Inheritance

Public - contains all the public components of all superclasses and its own added public components.

Protected - Contains all the protected components of all superclasses as well as its own.

Private - Contains its own private components. Address by its own methods.

Method Redefinition

Re-implement the method of super class

In Class declaration part:

Syntax:

METHODS meth REDEFINITION.

Up Cast

Up Cast (Also referred to as widening cast)

- Assignment between reference variables, where the static type of the target variables is similar or identical to the static type of the source variables.

Casting operator

*= or **MOVE ... TO ...***

Down Cast

Down Cast (Also called a narrowing cast)

- an assignment between reference variables in which the static type of the target variable more specific than the static type of the source variable.

Casting operator

?= or MOVE ... ?TO

Abstract Classes and Methods

When needed a class model for subclasses and do not require any objects from this class.

Abstract Class

Syntax:

CLASS class DEFINITION ABSTRACT.

...

ENDCLASS

Abstract Method

```
CLASS class DEFINITION ABSTRACT.  
    PUBLIC SECTION.  
        METHODS: method ABSTRACT.  
ENDCLASS.
```

Final Classes and Methods

When needed to protect a class or individual method from uncontrolled specialization

Final Class

Syntax:

CLASS class DEFINITION FINAL.

...

ENDCLASS

Final Method

Syntax:

CLASS class DEFINITION FINAL.

PUBLIC SECTION.

METHODS: method FINAL.

ENDCLASS

Constructors in Inheritance

Must always take the definition of the super classes into account

METHOD constructor.

CALL METHOD super->constructor.

ENDMETHOD.

Interface

- Describes the public visibility section of a given class without itself implementing functionality.
- Are defined either locally or globally in the class library.

Interface

- Resembles regular inheritance.
- Interfaces are simply superclasses that cannot be instantiated.
- Do not have an implementation part.
- Only have **public** components.

Define Interface

INTERFACE intf.

DATA ...

CLASS-DATA ..

METHODS ...

CLASS-METHODS ...

EVENTS ...

CLASS-EVENTS...

ENDINTERFACE.

Implement Interfaces in Classes

CLASS class DEFINITION.

PUBLIC SECTION.

...

INTERFACES: intf1, intf2 ...

...

ENDCLASS.

Interfaces are incorporated solely in the public visibility section of a class.

Interface Implementation in Classes

Class must implement all the methods of all incorporated interfaces in its implementation part.

CLASS class IMPLEMENTATION.

...

METHOD intf1~ method...

...

ENDMETHOD.

ENDCLASS.

Composing Interfaces

INTERFACE intf1.
 METHODS meth.
ENDINTERFACE.

INTERFACE intf2.
 INTERFACES intf1.
 METHODS meth.
ENDINTERFACE.

Composing Interfaces

CLASS class DEFINITION.

PUBLIC SECTION.

INTERFACES intf2.

ENDCLASS.

CLASS class IMPLEMENTATION.

METHOD intf1~meth. ... ENDMETHOD.

METHOD intf2~meth..... ENDMETHOD.

ENDCLASS.

Alias Names for Interface Component

Syntax

CLASS class DEFINITION.

PUBLIC SECTION.

INTERFACES intf.

ALIASES name FOR intf~comp.

...

ENDCLASS.

Alias Names for Interface Component

Example:

CALL METHOD oref~intf->method.

=

*CALL METHOD oref->alias.
oref->alias().*

Friend

The additions `FRIENDS` of statement `CLASS` define other classes or interfaces as friends of the class and thus grant them access to their protected and private components.

Friend - Example

```
INTERFACE i1.
```

```
...
```

```
ENDINTERFACE.
```

```
CLASS c1 DEFINITION CREATE PRIVATE FRIENDS i1.
```

```
PRIVATE SECTION.
```

```
DATA a1 TYPE c LENGTH 10 VALUE 'Class 1'.
```

```
ENDCLASS.
```

```
CLASS c2 DEFINITION.
```

```
PUBLIC SECTION.
```

```
INTERFACES i1.
```

```
METHODS m2.
```

```
ENDCLASS.
```

Friend - Example

CLASS c2 IMPLEMENTATION.

METHOD m2.

DATA oref TYPE REF TO c1.

CREATE OBJECT oref.

WRITE oref->a1.

ENDMETHOD.

ENDCLASS.

Unit 6: ABAP Workbench Tool

ABAP Workbench Tool

Class Builder

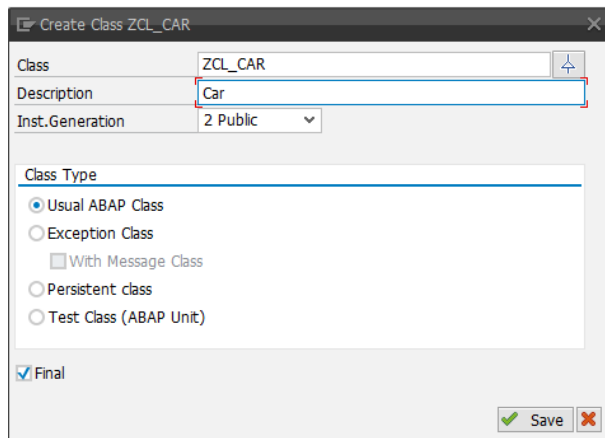
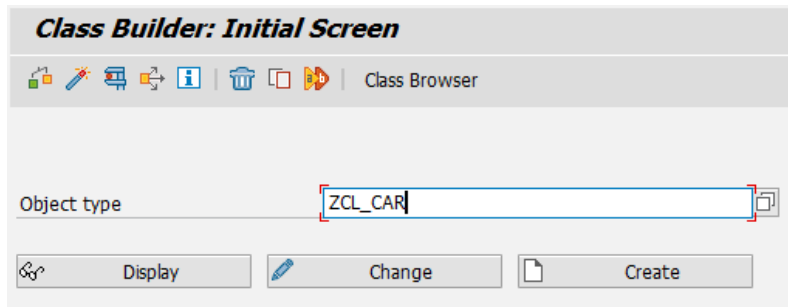
Global class is created with the Class Builder tool and visible in any ABAP program.

Class Builder is a special maintenance tool in ABAP Workbench

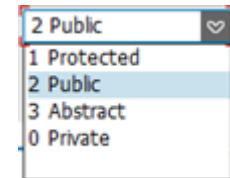
Class Builder

- ใช้ Class Builder (Transaction SE24) ในการสร้าง Global classes หรือ Interfaces
- โปรแกรม ABAP สามารถเรียกใช้ Global Classes ได้

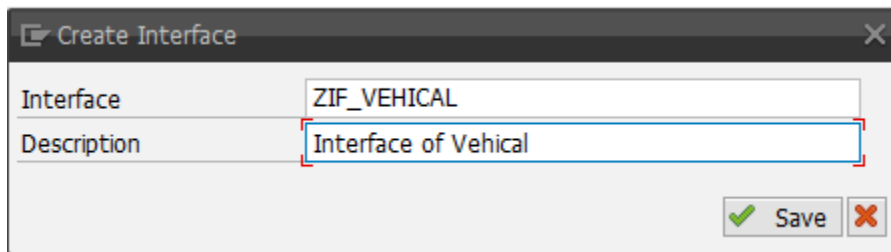
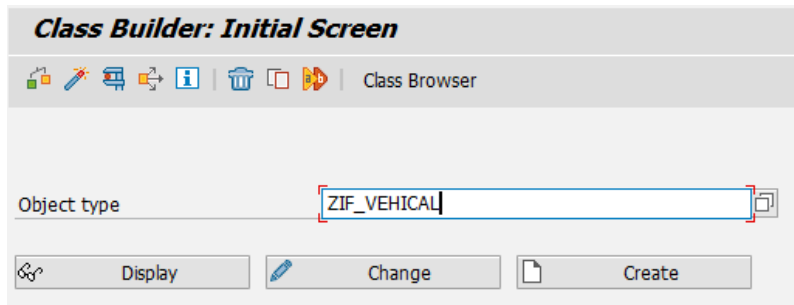
Create Global Classes



1. เข้า T-Code : SE24
2. Create class ขึ้นต้นด้วย ZCL_* หรือ YCL_*
3. กำหนด Description
4. เลือกประเภทของ Class
5. กำหนดเป็น Final Classes ได้
6. Save



Create Interfaces



1. เข้า T-Code : SE24
2. Create interface ขึ้นต้นด้วย ZIF_* หรือ YIF_*
3. Save

Global Classes Vs Interfaces

Class Builder: Change Class ZCL_CAR

Class/Interface: ZCL_CAR Implemented / Inactive

Properties Interfaces Friends Attributes Methods Events Types Aliases

Superclass Undo inheritance Change Inheritance

Description: Car

Inst. Generation: 2 Public

☒ Final

General Data

Message Class	
Program status	
Category	0 General Object Type
Package	\$TMP
Original Language	EN
Created	PUI 01.03.2016
Last change	PUI 01.03.2016

☐ Shared Memory-Enabled ☒ Fixed point arithmetic ☒ Unicode checks active

Forward declarations

Type group/Object type

Global Classes

Class Builder: Change Interface ZIF_VEHCAL

Interface: ZIF_VEHCAL Implemented / Inactive

Properties Interfaces Attributes Methods Events Types Aliases

Description: Interface of Vehcal

General Data

Package	\$TMP
Original Language	EN
Created by	PUI 01.03.2016
Last changed by	PUI 01.03.2016

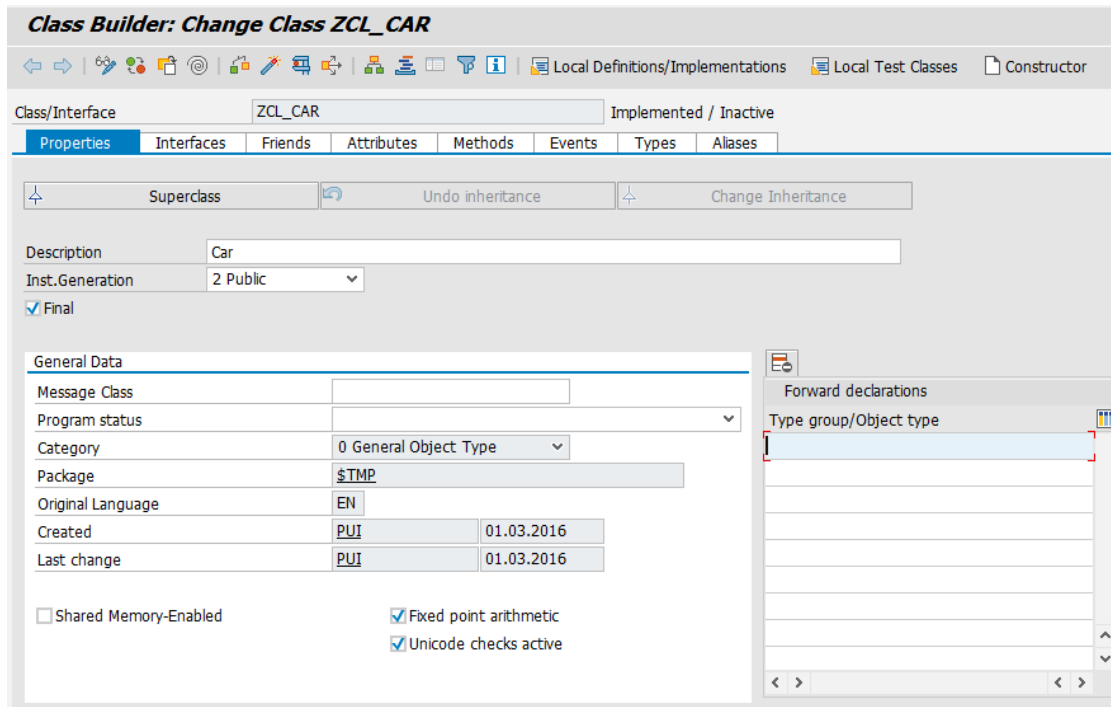
☒ Unicode checks active

Forward declarations

Type group/Object type

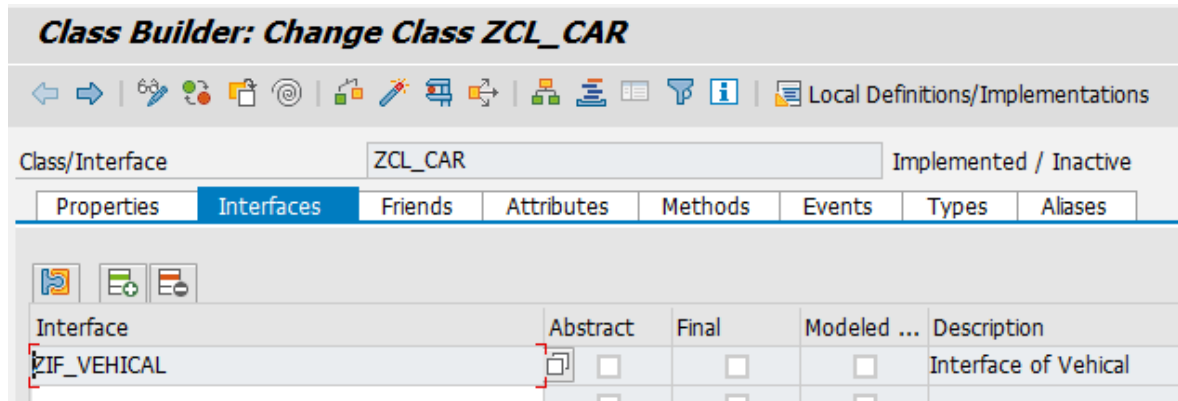
Interfaces

Properties (Global Classes)



- สามารถทำการ **Inherit class** ได้ (Superclass)
- กำหนดให้เป็น **Final Classes**
- กำหนด **Message Class** จาก **SE91** ได้
- Include **Type group/Object type**

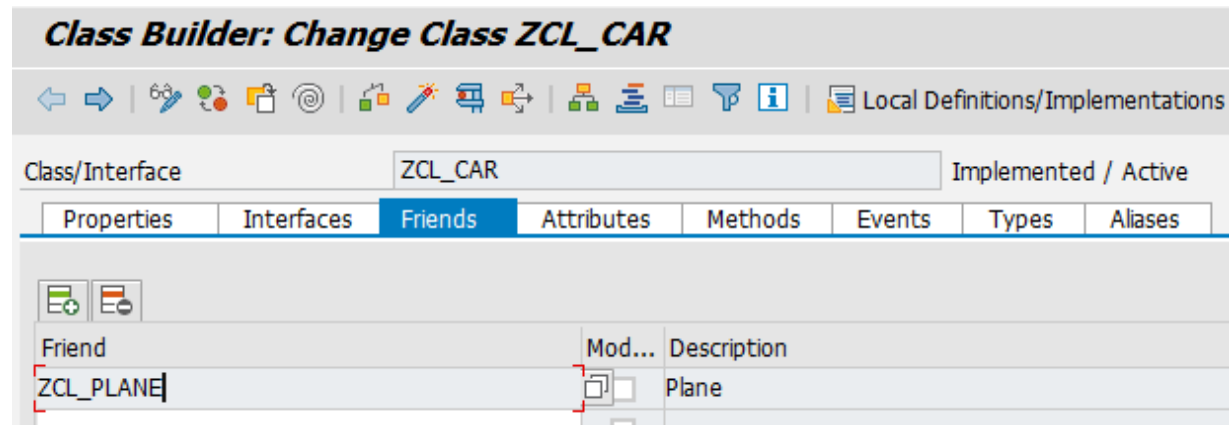
Interfaces (Global Classes)



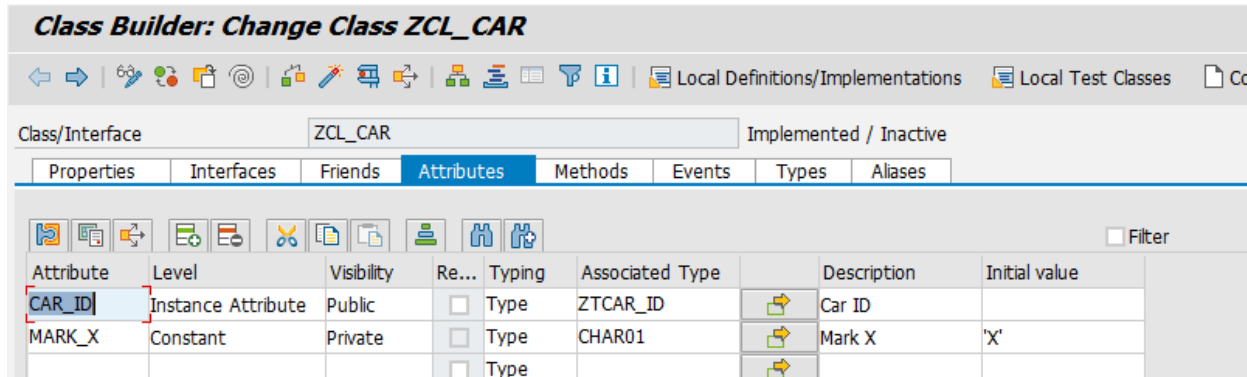
- ใส่ชื่อ Interface ที่ต้องการนำมาใช้งานใน Class
- Class สามารถใช้ attributes, events, constants หรือ types ที่อยู่ใน Interface ตามที่ประกาศไว้ได้
- ส่วนประกอบของ Interface ที่ถูกนำมาใช้ใน Class จะมีชื่อ <interface name>~<component name>

Friends (Global Classes)

- สามารถใช้เทคนิค FRIENDs concept ได้



Attributes (Global Classes)



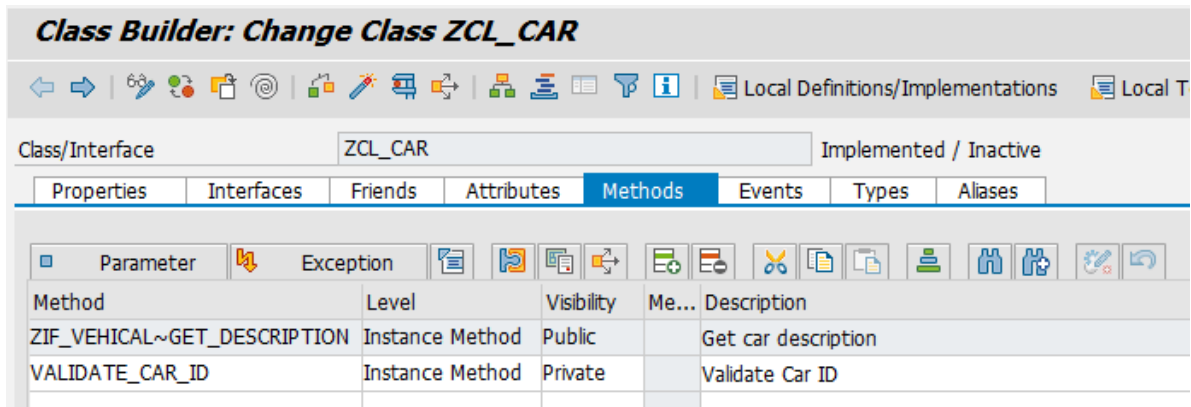
- ประกาศตัวแปร และกำหนดค่าคงที่ได้
- Level** กำหนดประเภทของ **Attribute**

NUM	Description
0	Instance Attribute
1	Static Attribute
2	Constant

- Visibility** กำหนดการเข้าถึง **Attribute**

NUM	Description
2	Public
1	Protected
0	Private

Methods (Global Classes) - (1)

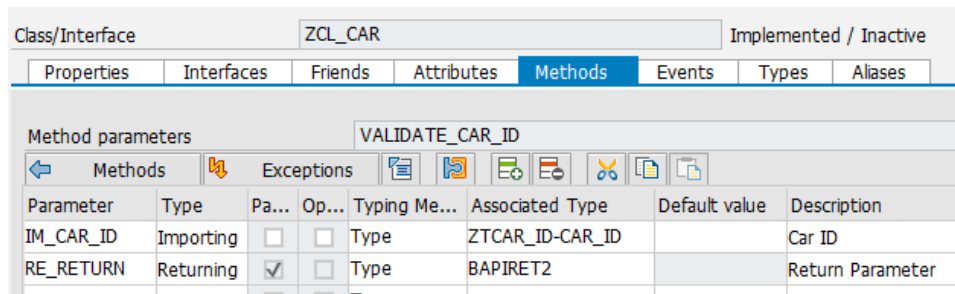
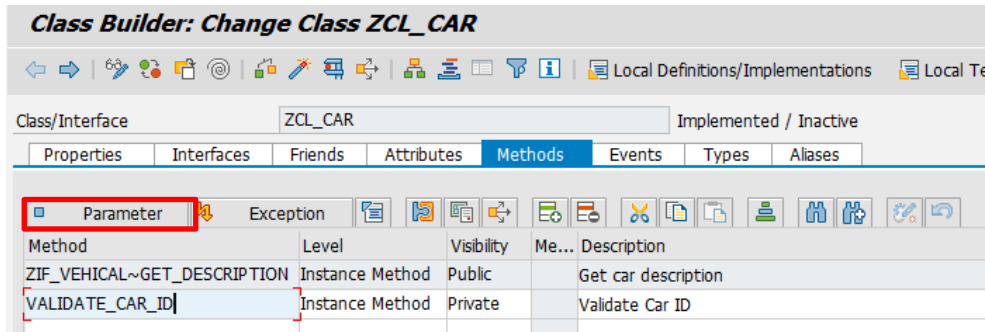


- สร้าง Method
- Level กำหนดประเภทของ Method
- Visibility กำหนดการเข้าถึง Method

NUM	Description
0	Instance Method
1	Static Method

NUM	Description
2	Public
1	Protected
0	Private

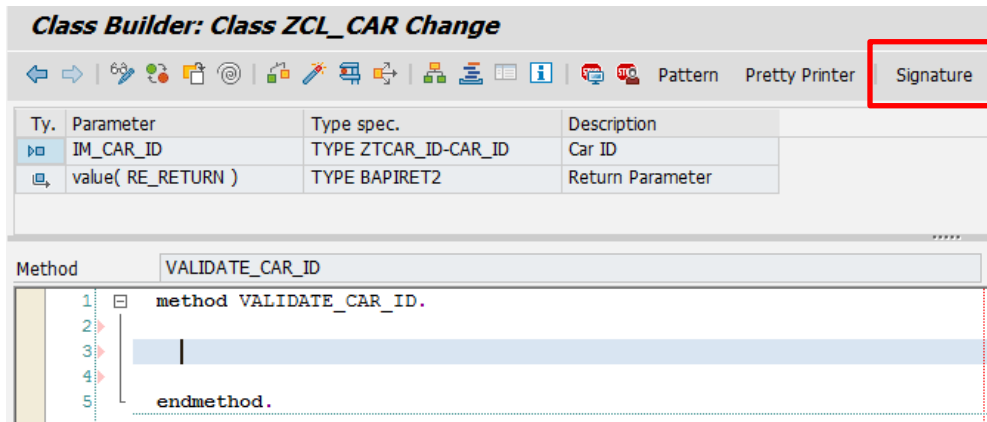
Methods (Global Classes) - (2)



- กำหนด Parameter ให้กับ Method
- ประเภทของ Parameter

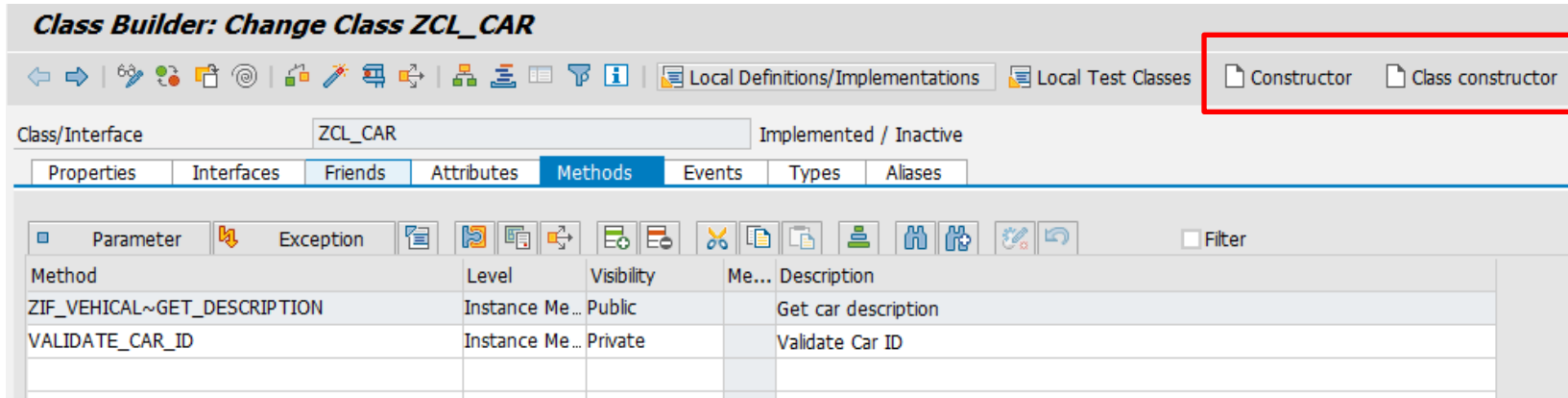
NUM	Description
0	Importing
1	Exporting
2	Changing
3	Returning

Methods (Global Classes) - (3)



- พื้นที่เขียน **Code** ภายใน **Method**
- แสดง **Parameter** ของ **Method** ที่ด้านบน โดยสามารถกดปุ่ม **Signature** เพื่อเปิด/ปิดการแสดงผล **Parameter** ได้

Methods (Global Classes) - (4) - Constructors



- **Constructor** คือ Instance Method ที่ทำงานเป็น Method แรกทันที ที่ทำการ Create instance (CREATE OBJECT, NEW)
- **Class constructor** คือ Static Method ที่ทำงานเป็น Method แรกทันทีที่มีการเรียกใช้งาน Class (Create instance , Call Static)

Events (Global Classes) - (1)

Class Builder: Change Class ZCL_CAR

Class/Interface: ZCL_CAR Implemented / Inactive

Properties Interfaces Friends Attributes Methods **Events** Types Aliases

Parameters

Event	Type	Visibility	Description
OWNER	Instance Event	Public	

1. OWNER
2. Parameters



Class/Interface: ZCL_CAR Implemented / Inactive

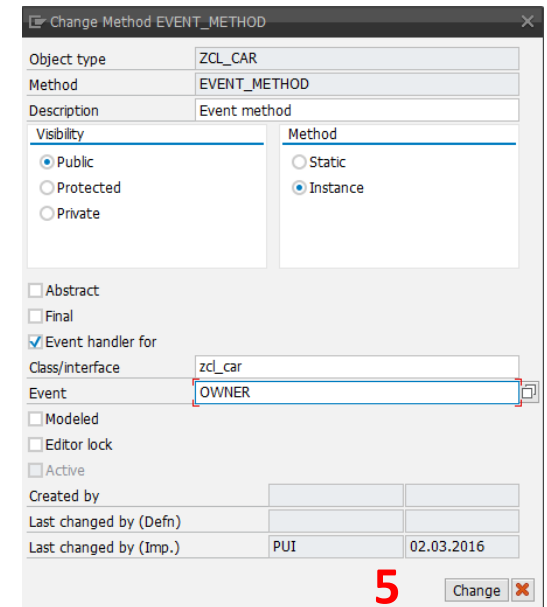
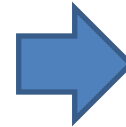
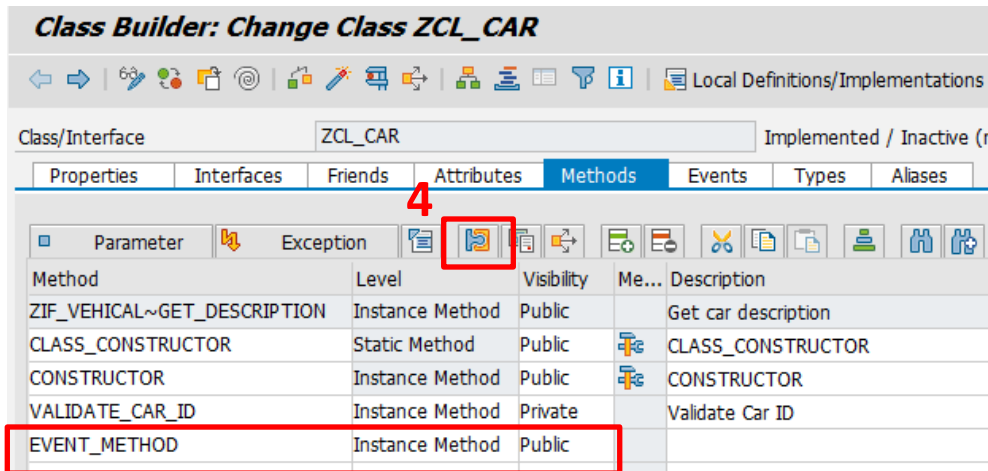
Properties Interfaces Friends Attributes Methods **Events** Types Aliases

Event parameters: OWNER

Parameters	Op...	Typing	Associated Type	Default value	Description
IM_OWNER_ID	☐	Type	ZTCAR_OWNER-OWNER_ID		Owner ID

1. Create Event
2. กำหนด Parameters สำหรับ Event

Events (Global Classes) - (2)

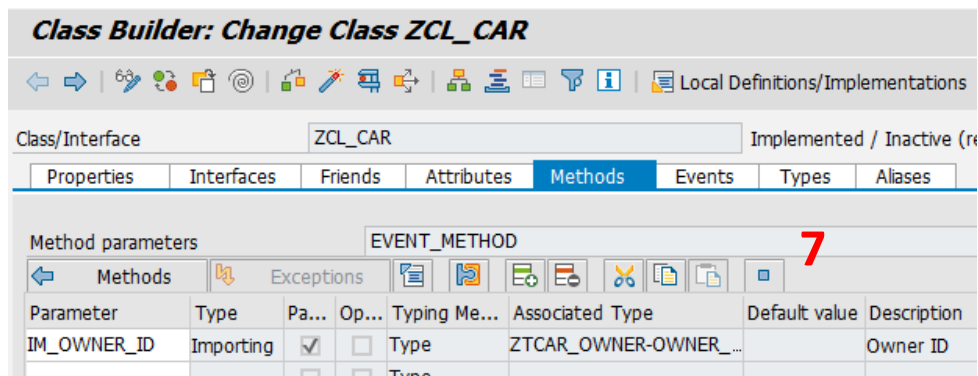
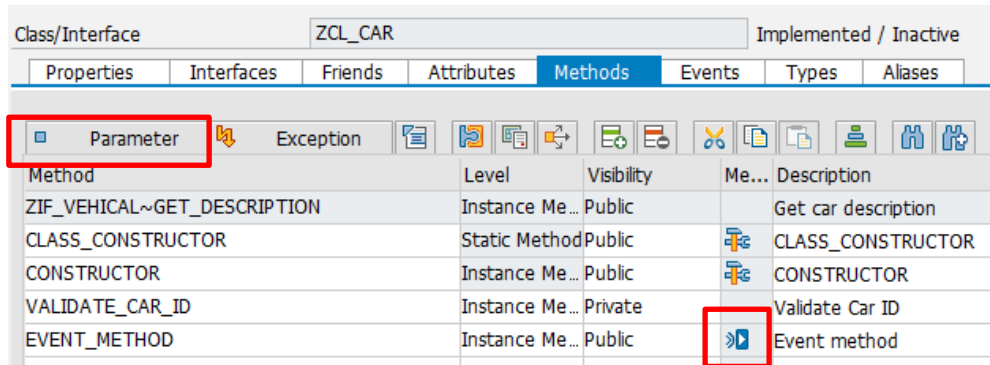


3. Create Method สำหรับ Event

4. กดปุ่ม Detail View 

5. กำหนด Event จาก Tab Event ที่ต้องการเชื่อมกับ Method แล้วกดปุ่ม Change

Events (Global Classes) - (3)



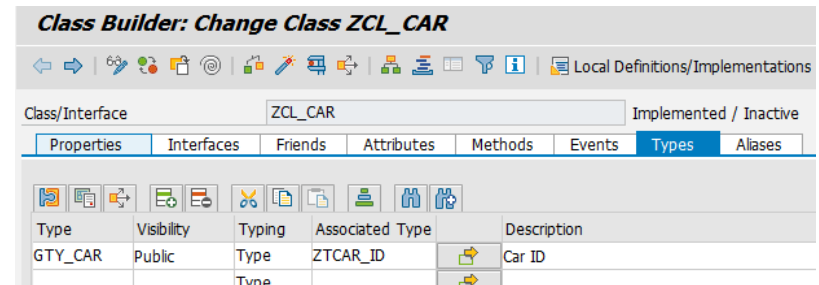
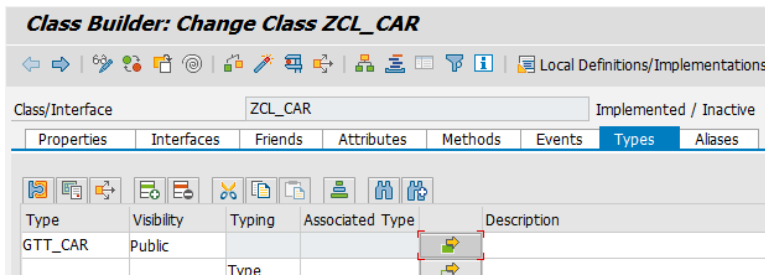
- จะได้สัญลักษณ์ Event Handler



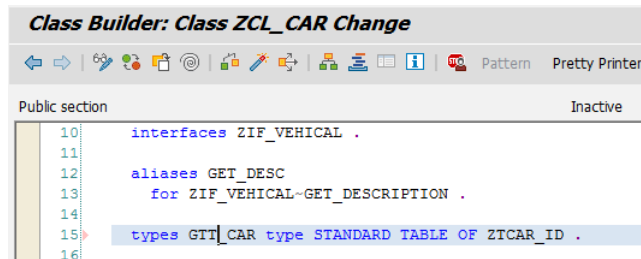
6. กด Parameter ของ Method Event 
7. กดปุ่ม Copy event parameters

Types (Global Classes)

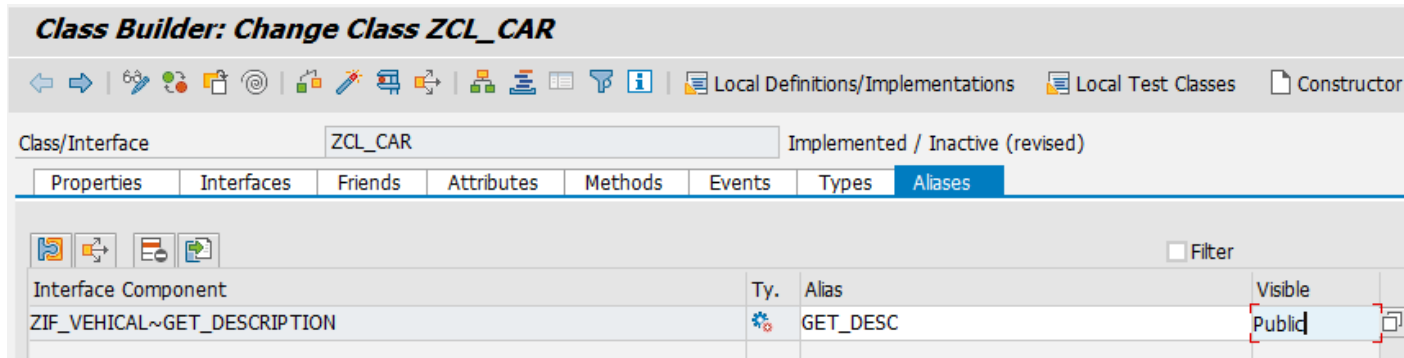
- ประกาศ Type ภายใน Class
- ประกาศแบบ Global Type



Global Type



Aliases (Global Classes)



- ใช้สำหรับ Interfaces เพื่อตั้งชื่อแทน Interface Component
- ตย. ใช้ CALL METHOD OBJ->get_desc แทนการใช้
CALL METHOD OBJ->ZIF_VEHICAL~GET_DESCRIPTION

Unit 7: OO Transaction

OO Transaction

OO Transactions

In transaction maintenance (SE93), you can specify a transaction code as an OO transaction.

OO Transactions

```
Subroutine Pool: DEMO_OO_TRANSACTION Active

1  *&-----*
2  *& Subroutine pool DEMO_OO_TRANSACTION *
3  *& *
4  *&-----*
5
6  PROGRAM demo_oo_transaction.
7
8  *
9
10 CLASS demo_class DEFINITION.
11     PUBLIC SECTION.
12         METHODS instance_method.
13     ENDClass.
14
15     *
16
17 CLASS demo_class IMPLEMENTATION.
18     METHOD instance_method.
19         cl_demo_output=>display( 'Instance method in local class' ).
20     ENDMETHOD.
21 ENDClass.
```

OO Transactions

The screenshot shows the SAP transaction code configuration screen for `DEMO_OO_METHOD`. The package is `SABAPDEMOS` and the transaction text is "OO Trans. for Local Instance Method". Under "Start Options", the "OO transaction model" checkbox is unchecked. The "Class Name" is `DEMO_CLASS` and the "Method" is `:INSTANCE_METHOD`. The "Local in program" checkbox is checked, and the transaction name is `DEMO_OO_TRANSACTION`. A red double-headed arrow points to the "Method" field.

Transaction code: DEMO_OO_METHOD

Package: SABAPDEMOS

Transaction text: OO Trans. for Local Instance Method

Start Options

☐ OO transaction model

Class Name: DEMO_CLASS

Method: :INSTANCE_METHOD

☒ Local in program

DEMO_OO_TRANSACTION

Output

Instance method in local class

The Control Framework

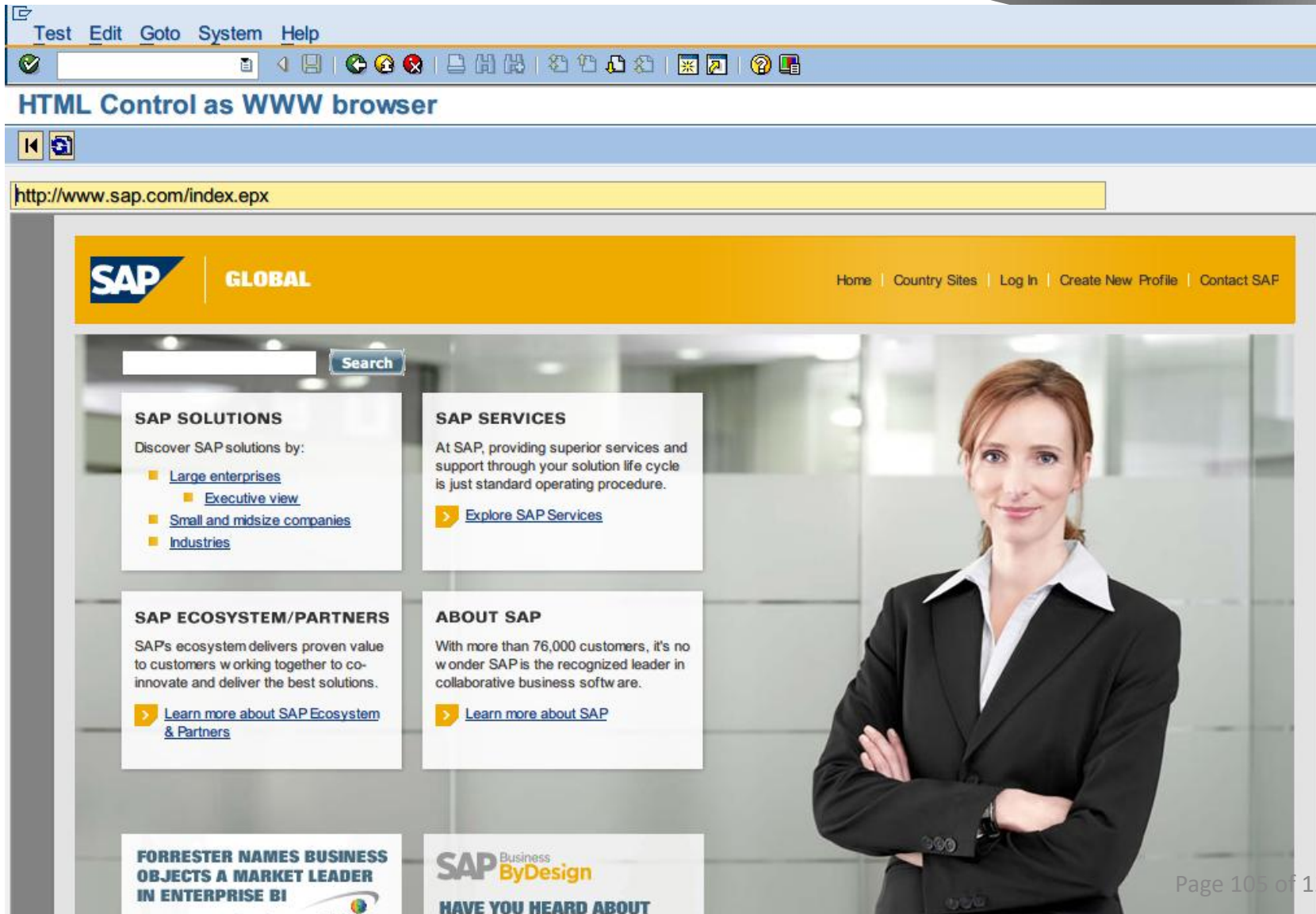
The Control Framework (CFW)

In the SAP system, you can use ABAP to control desktop applications (custom controls).

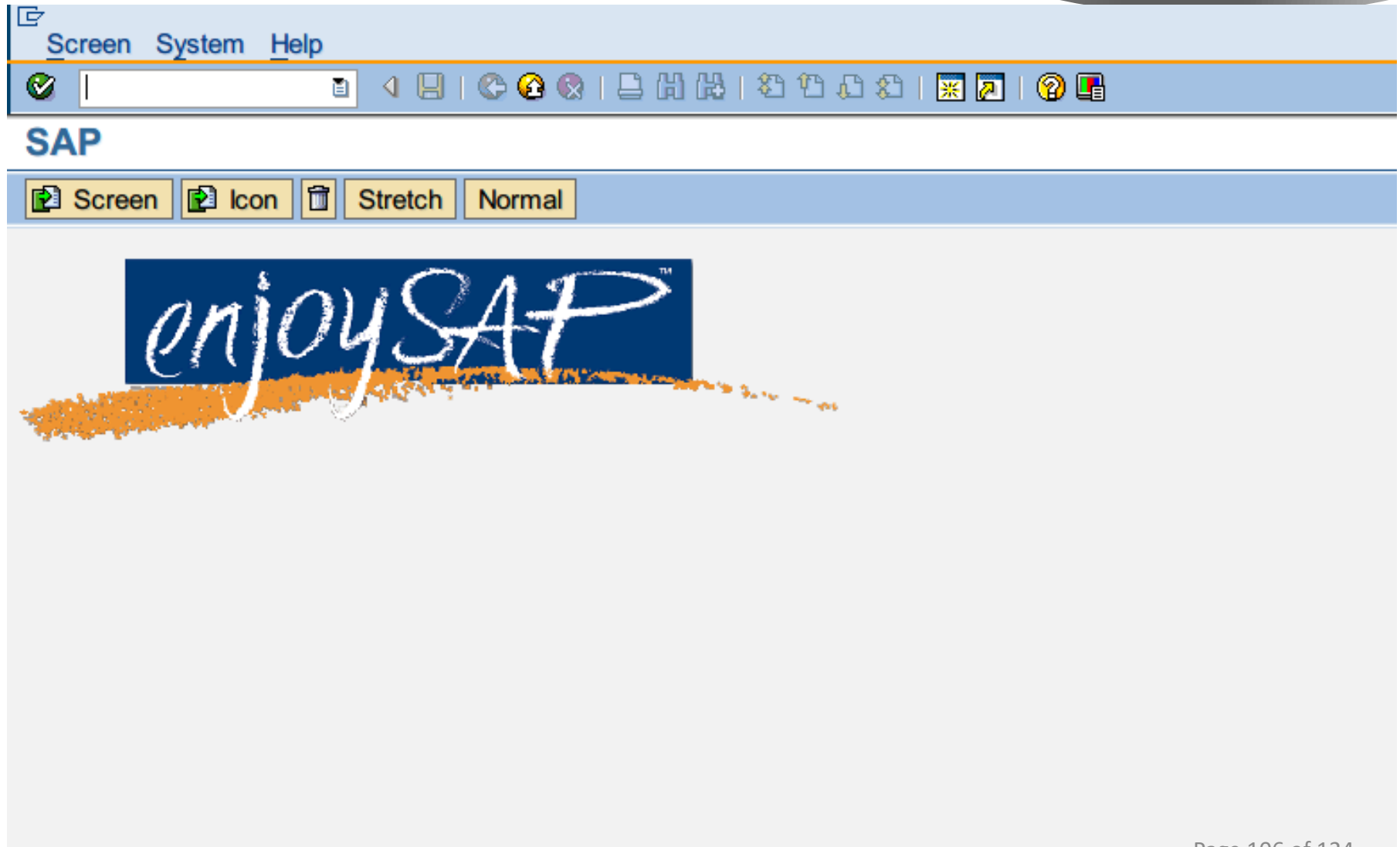
The application logic runs on the SAP application server, which drives the custom control at the frontend.

The Control Framework supports controls (ActiveX and JavaBeans) that are implemented within the SAP GUI.

CFW – HTML Control



CFW – Picture Control



CFW – ALV Grid Control

 List Edit Goto Settings System Help 								
								
Airli...	Flight...	Flight Date	Airfare	Curren...	Plane Type	Capacity	Occupied econ.	
AA	17	2008/02/13	423.94	USD	146-200	385	375	
AA	17	2008/02/14	422.94	USD	146-200	0	0	
AA	17	2008/03/12	422.94	USD	747-400	385	372	
AA	17	2008/04/09	422.94	USD	747-400	385	374	
AA	17	2008/05/07	422.94	USD	747-400	385	371	
AA	17	2008/06/04	422.94	USD	747-400	385	373	
AA	17	2008/07/02	422.94	USD	747-400	385	373	
AA	17	2008/07/30	422.94	USD	747-400	385	364	
AA	17	2008/08/27	422.94	USD	747-400	385	368	
AA	17	2008/09/24	422.94	USD	747-400	385	53	
AA	17	2008/10/22	422.94	USD	747-400	385	122	
AA	17	2008/11/19	422.94	USD	747-400	385	90	
AA	17	2008/12/17	422.94	USD	747-400	385	36	
AA	17	2009/01/14	422.94	USD	747-400	385	42	
AA	17	2009/02/11	422.94	USD	747-400	385	36	
AA	64	2008/02/15	422.94	USD	A310-300	280	271	
AA	64	2008/03/14	422.94	USD	A310-300	280	271	
AA	64	2008/04/11	422.94	USD	A310-300	280	269	
AA	64	2008/05/09	422.94	USD	A310-300	280	260	

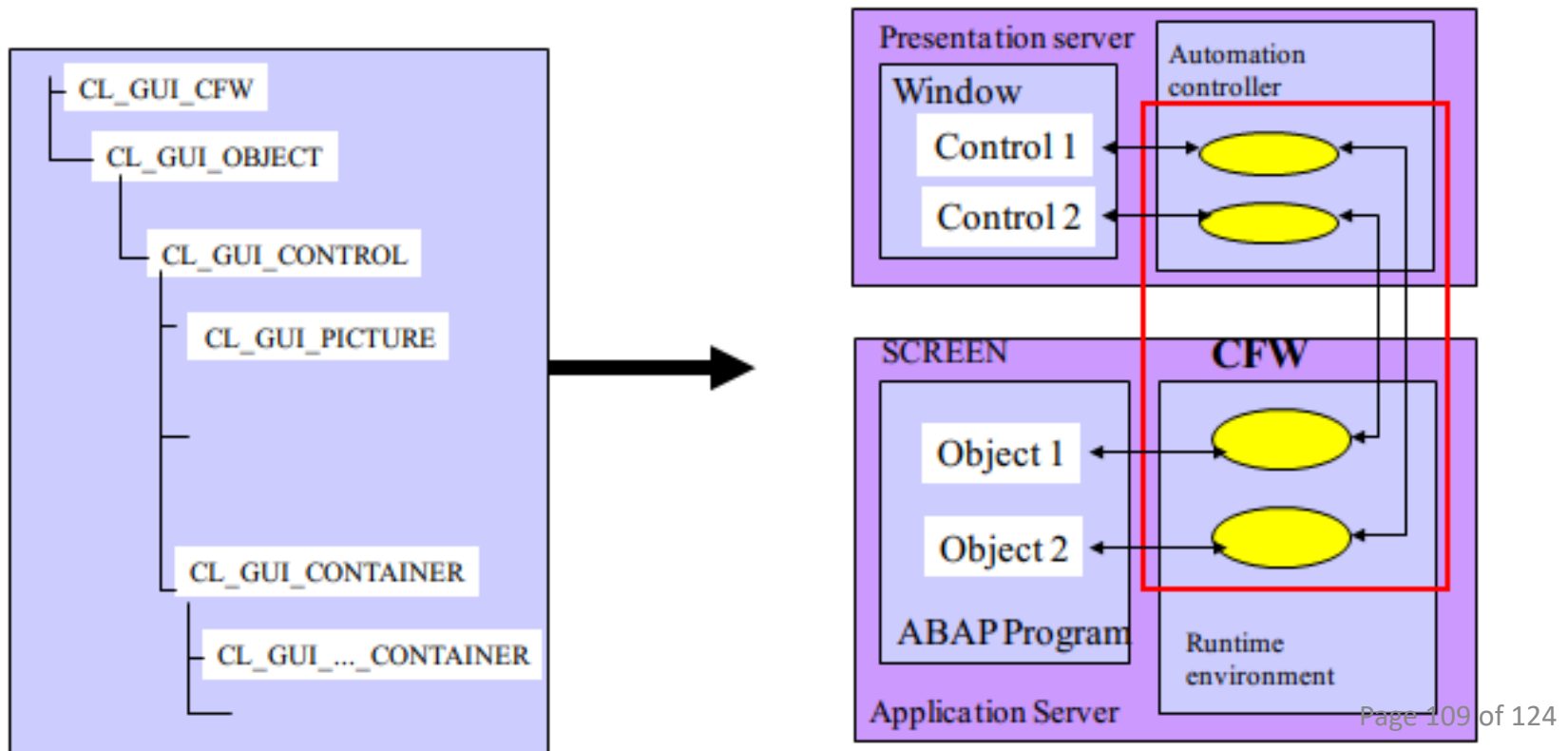
Control Framework Architecture

Global class in the class library for each GUI control supplied and that when working with GUI controls, for each control object on the frontend there is a proxy object of the corresponding class in the ABAP programs.

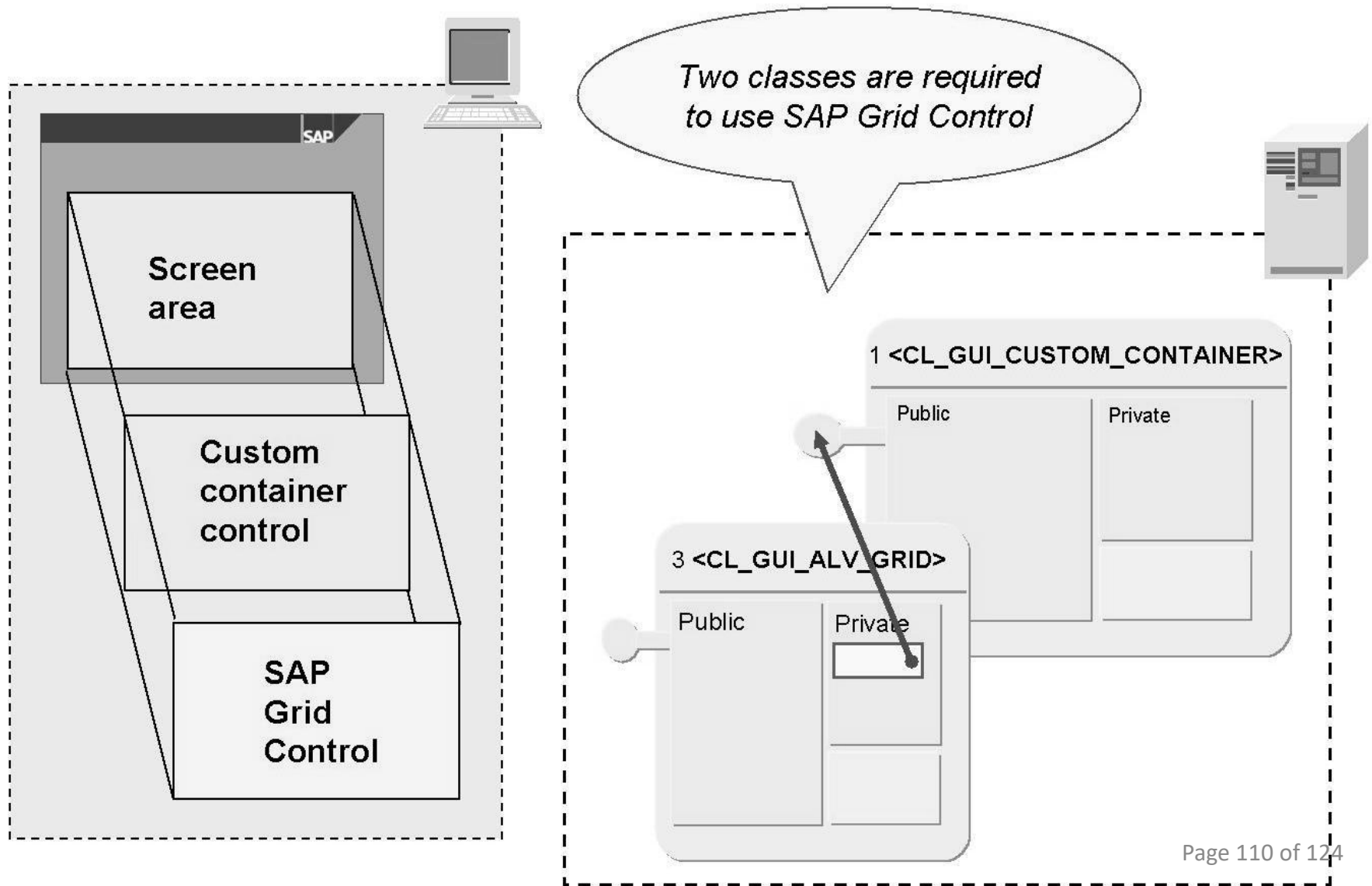
Control Framework Architecture

ABAP programming works only with the proxy objects

The CFW forwards its method calls to actual GUI control objects and informs the objects of the ABAP program by triggering events about the user actions on the GUI controls.

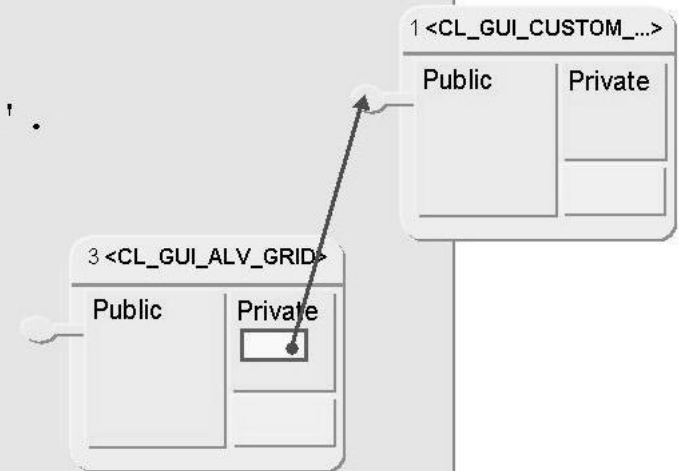


Control Framework Architecture



Control Framework Example

```
...  
DATA: container_r TYPE REF TO cl_gui_custom_container,  
      grid_r      TYPE REF TO cl_gui_alv_grid.  
...  
CREATE OBJECT container_r  
      EXPORTING container_name = 'CONTAINER_1'.  
...  
CREATE OBJECT grid_r  
      EXPORTING i_parent = container_r.  
...  
grid_r->set_table_for_first_display(  
    i_structure_name = 'SAPLANE'  
    it_outtab       = itab_saplane ).  
...
```



SAP

alv_grid_control

AA	17	2000-01-17	USD	515,69	AS 21
AA	17	2000-02-20	USD	515,69	AS 21
AA	17	2000-03-11	USD	515,69	747-400
AA	64	2000-05-19	USD	369,00	747-400
LH	400	2000-01-15	DEM	1234,56	AS 10-200
LH	400	2000-02-26	DEM	1234,56	AS 10-200
LH	400	2000-03-21	DEM	1234,56	AS 10-200
LH	402	2000-03-04	DEM	1234,56	AS 19
LH	402	2000-05-28	DEM	1234,56	AS 19

Page 111 of 124

Container Controls

CL_GUI_CUSTOMER_CONTAINER

CL_GUI_DOCKING_CONTAINER

CL_GUI_SPLITTER_CONTAINER

CL_GUI_DIALOGBOX_CONTAINER

Application Controls

CL_GUI_TOOLBAR

CL_GUI_PICTURE

CL_GUI_HTML_VIEWER

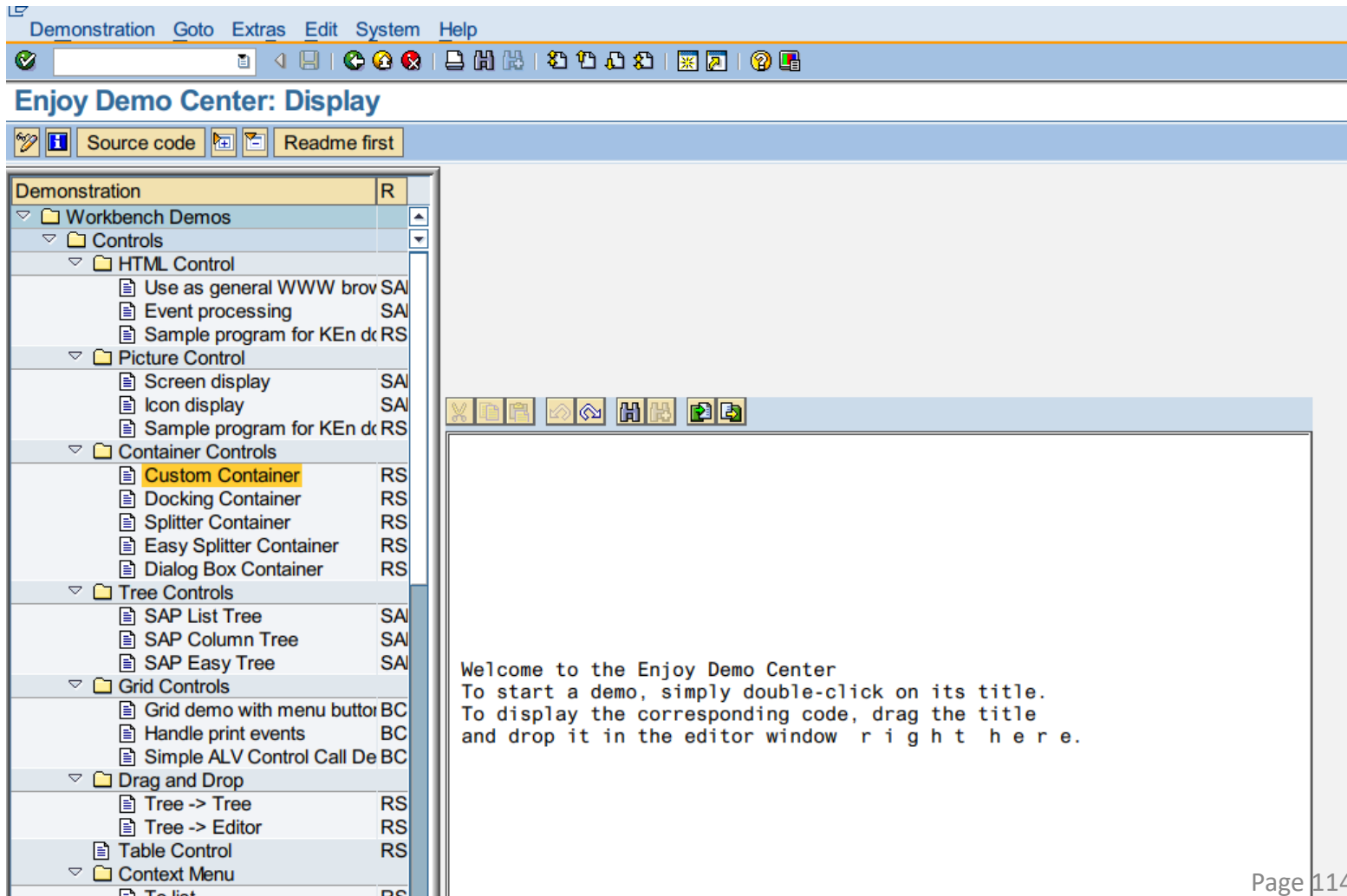
CL_GUI_TEXTEDIT

CL_GUI_SIMPLE_TREE

CL_GUI_ALV_GRID

Enjoy Demo Center

TCode: DWDM



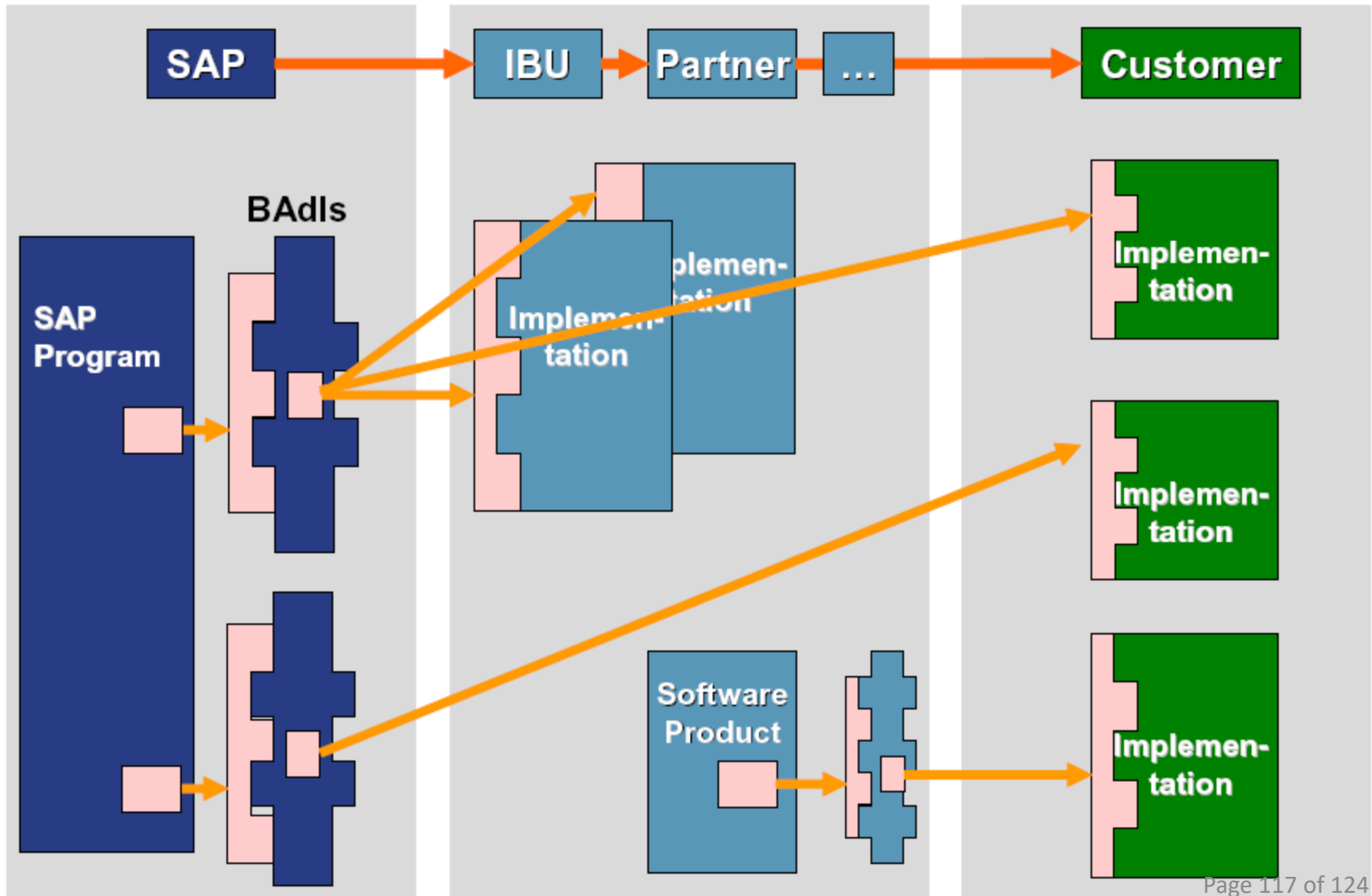
Unit 9: BAdIs

BAdIs

Enhancement using Business Add-Ins

Business Add-Ins (BAdIs) are one of the most important technologies used to adapt SAP software to specific requirements.

Business Add-Ins: Architecture



Implementation of Business Add-Ins

The screenshot displays the SAP Class Builder interface for the class **ZCL_IM_BC425_01FLIGHT2**. The menu bar includes **Method**, **Edit**, **Goto**, **Utilities**, **Environment**, **System**, and **Help**. The toolbar contains various icons for file operations and development tools. Below the toolbar, the title bar reads **Class Builder: Class ZCL_IM_BC425_01FLIGHT2 Display**. A secondary toolbar includes buttons for **Pattern**, **Pretty Printer**, **Signature**, **Public Section**, and **Protect**.

A table lists the parameters of the selected method:

Ty.	Parameter	Type spec.	Description
▢	VALUE(E_FLIGHT)	TYPE SFLIGHT01	Flight

The **Method** list shows **IF_EX_BC425_01FLIGHT2~GET_DATA** as the active method. The code editor displays the following implementation:

```
1 method IF_EX_BC425_01FLIGHT2~GET_DATA.  
2   MOVE-CORRESPONDING IF_EX_BC425_01FLIGHT2~wa to e_flight.  
3 endmethod.
```

Q & A